From Reward Generation to Reward Search: Experience-Guided Schema Adaptation with Semantic Attribution and Risk-Aware Editing

Anonymous

Abstract

LLM-assisted reward design has recently emerged as a promising way to reduce manual reward engineering in reinforcement learning. Existing methods show that large language models can generate and refine executable reward code from task descriptions and training feedback. However, reliable reward design often requires more than reward generation: a system must diagnose why a reward failed, remember which previous edits helped or hurt, connect policy behavior to reward components, and validate proposed edits under reward-hacking risk. We propose EG-RSA, an Experience-Guided Reward Schema Adaptation framework that formulates LLM-assisted reward design as history-aware reward schema search. EG-RSA represents rewards as structured schemas with semantic roles, attributes policy behavior to reward components, stores effective and regressive edits as outcome lessons, and constrains LLM modifications through auditable edit operators with risk-aware validation and rollback. We study EG-RSA on LunarLander-v3 as a mechanism-verification benchmark, analyzing effective edits, regression lessons, and the safety–exploration trade-off introduced by behavior risk audit.

1 Introduction

Reinforcement learning (RL) has achieved strong results in sequential decision-making and control tasks, but its success remains highly dependent on the reward function that defines the learning objective [Sutton and Barto, 2018]. Designing effective rewards is difficult: a reward must provide useful learning signals, encourage task completion, and remain aligned with the intended objective. Poorly specified rewards can lead to unintended behaviors, including reward hacking, excessive exploitation of dense shaping signals, or high proxy reward with poor true task performance [Amodei et al., 2016, Pan et al., 2022].

Recent work has explored large language models (LLMs) as automated reward designers. Eureka demonstrates that LLMs can generate executable reward code and improve it through iterative in-context optimization [Ma et al., 2023]. Text2Reward generates dense reward functions from natural language task descriptions [Xie et al., 2023]. Auto MC-Reward uses trajectory analysis and reward critics to refine rewards in sparse-reward Minecraft tasks [Li et al., 2023]. CARD further studies dynamic feedback and trajectory preference evaluation for LLM-driven reward design [Sun et al., 2024]. These methods show that LLMs can reduce the burden of manual reward engineering.

However, reliable reward design requires more than generating or refining reward code. When a reward fails, the key question is often not only how to rewrite it, but why it failed: which reward component dominated policy behavior, which previous edit improved or degraded the policy, and whether a new edit resembles a prior harmful search direction. Existing LLM-based reward design methods provide valuable feedback loops, but they do not explicitly model reward design as a structured search process over accumulated editing experience.

We argue that LLM-assisted reward design should be formulated as experience-guided reward schema search. Instead of repeatedly generating free-form reward code, the system should maintain a structured reward schema across iterations, diagnose policy failures through semantic reward roles, store effective and regressive edits as reusable outcome lessons, and validate each LLM-proposed edit before applying it.

We propose EG-RSA, an Experience-Guided Reward Schema Adaptation framework for LLM-assisted reinforcement learning. EG-RSA represents rewards as componentized schemas with semantic roles such as dense guidance, stability quality, terminal success, safety constraint, and control cost. After each training iteration, EG-RSA collects trajectory feedback, computes semantic outcomes, attributes policy behavior to reward roles, retrieves relevant outcome lessons, and asks an LLM to propose an operator-constrained reward edit. The edit is then checked by scale validation, behavior risk audit, repair, and rollback mechanisms.

Our contributions are threefold:

1. We formulate LLM-assisted reward design as experience-guided reward schema search, where rewards are maintained as structured, versioned schemas rather than regenerated as unconstrained code.
2. We introduce semantic role attribution and outcome lesson memory to connect policy failures with reward components and reuse effective or regressive reward-edit experiences across iterations.
3. We design a risk-aware operator-constrained editing loop with validation and rollback, and analyze the safety–exploration trade-off that arises when audit rules are too conservative in low-success regimes.

2 Related Work

2.1 Reward Design and Reward Shaping

Reward design is a long-standing challenge in reinforcement learning. Since the reward function defines the optimization objective, learned policies are highly sensitive to how task intent is translated into scalar feedback [Sutton and Barto, 2018]. Reward shaping aims to improve learning efficiency by adding auxiliary signals, and potential-based shaping provides conditions under which reward transformations preserve optimal policies [Ng et al., 1999]. However, these results also highlight the difficulty of reward engineering: shaping signals must be designed carefully to accelerate learning without changing the intended behavior.

Another line of work studies reward learning from demonstrations. Inverse reinforcement learning infers reward functions from expert behavior [Ng and Russell, 2000], and apprenticeship learning uses such inferred objectives to imitate expert policies [Abbeel and Ng, 2004]. These methods reduce manual reward specification but typically require demonstrations or assumptions about expert behavior. EG-RSA addresses a different setting: it does not assume expert demonstrations, but adapts an initial reward schema through policy training feedback, semantic diagnosis, and accumulated reward-edit experience.

2.2 LLM-Assisted Reward Design

Large language models have recently been used to automate reward design. Eureka demonstrates that LLMs can generate executable reward code and improve it through iterative in-context optimization [Ma et al., 2023]. Text2Reward generates shaped dense rewards from natural language task descriptions [Xie et al., 2023]. Auto MC-Reward combines a Reward Designer, Reward Critic,

and Trajectory Analyzer to refine dense rewards from trajectory feedback [Li et al., 2023]. CARD further introduces dynamic feedback and trajectory preference evaluation to improve the efficiency of LLM-driven reward refinement [Sun et al., 2024]. A recent survey summarizes LLM-enhanced RL and identifies reward design as a key role for LLMs [Cao et al., 2024].

These methods establish that LLMs can be effective reward designers. EG-RSA follows this direction but focuses on a different question: how can reward-search experience be stored and reused across iterations? Instead of treating each reward update as an isolated code generation step, EG-RSA maintains a versioned reward schema, attributes policy behavior to semantic reward roles, records effective and regressive edits as outcome lessons, and restricts LLM modifications to auditable operators. This turns reward design from reward code generation into structured, history-aware reward schema search.

2.3 Memory, Reflection, and Reliability in Reward Search

EG-RSA is related to memory-based and reflection-based language agents. Reflexion improves language agents through verbal feedback and episodic memory without updating model parameters [Shinn et al., 2023]. Voyager maintains a skill library for lifelong embodied learning [Wang et al., 2023], while Generative Agents use memory streams and reflection to support long-horizon behavior [Park et al., 2023]. These works show that LLM agents can benefit from storing and reusing past experience. EG-RSA applies this idea to reward design, but stores structured reward-edit outcome lessons rather than natural language reflections or executable action skills.

Reliability is also central to reward design. Reward hacking and reward misspecification occur when an agent optimizes a proxy reward while failing the intended objective [Amodei et al., 2016, Pan et al., 2022, Skalse et al., 2022]. This concern is especially relevant for LLM-generated rewards, where dense shaping terms may be easy to optimize but misaligned with sparse terminal success. EG-RSA addresses this problem through failure-mode diagnosis, semantic role attribution, behavior risk audit, and rollback. At the same time, our analysis shows that safety mechanisms must be balanced with exploration: if risk audit is too conservative, it can block the edits needed to escape low-success regimes.

3 Method

3.1 Problem Formulation and Framework Overview

We consider an RL environment in which a policy is trained using a reward function compiled from a structured reward schema. Let S_k denote the reward schema at iteration k, and let R_{S_k} be the executable reward function compiled from that schema. EG-RSA does not directly optimize the policy parameters as its main contribution; instead, it searches for a sequence of reward schemas $\{S_k\}_{k=0}^{K}$ such that policies trained under the compiled rewards increasingly align with the intended task objective.

Given an initial reward schema S_0, a task description, diagnostic specifications, and a memory store, EG-RSA performs iterative reward schema search. In each iteration, it compiles the current schema into reward code, trains a policy with the compiled reward, collects semantic feedback and trajectory diagnostics, attributes policy behavior to reward roles, retrieves relevant historical lessons, asks an LLM edit agent to propose a constrained edit plan, and validates the edit through risk-aware audit and rollback.

Figure 1 illustrates the framework. The key difference from one-shot reward generation is that EG-RSA maintains a versioned reward schema and a memory of prior edit outcomes across itera-

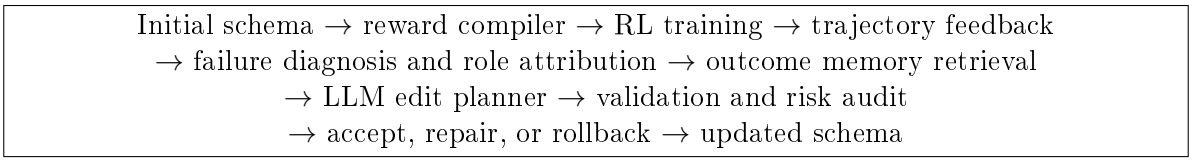

Figure 1: Overview of one EG-RSA reward schema search iteration.

tions. The LLM is therefore used as a reward search operator rather than a free-form reward-code generator.

3.2 Reward Schema and Edit Operators

EG-RSA represents a reward function as a reward schema

$$S = (\mathcal{C}, \mathcal{E}, \mathcal{M}), \tag{1}$$

where $\mathcal{C} = \{c_i\}_{i=1}^{n}$ is a set of dense reward components, $\mathcal{E} = \{e_j\}_{j=1}^{m}$ is a set of event-based reward rules, and $\mathcal{M}$ contains metadata such as semantic roles. Each dense component has a name, a weight, a metric function, and a semantic role. Typical roles include `dense_guidance`, `stability_quality`, `terminal_success`, `safety_constraint`, and `control_cost`.

The reward function compiled from schema S can be written as

$$R_S(s_t, a_t, s_{t+1}) = \sum_{i=1}^{n} w_i \phi_i(s_t, a_t, s_{t+1}) + \sum_{j=1}^{m} \beta_j \mathbf{1}[e_j(s_t, a_t, s_{t+1})], \tag{2}$$

where ϕ_i is a dense metric function, w_i is its weight, e_j is an event condition, β_j is the event reward, and $\mathbf{1}[\cdot]$ is an indicator function.

Instead of allowing the LLM to freely rewrite the full reward function, EG-RSA restricts reward modification to an operator-constrained edit plan

$$\Delta_k = \{o_1, o_2, \ldots, o_p\}, \tag{3}$$

where each operator belongs to a predefined set, including `increase_weight`, `decrease_weight`, `add_component`, `add_event_rule`, and `disable_component`. A candidate schema is obtained by

$$S'_k = \mathrm{Apply}(S_k, \Delta_k). \tag{4}$$

This design makes edits inspectable, reversible, and auditable.

3.3 Semantic Feedback and Role Attribution

After training a policy with the current schema, EG-RSA evaluates policy behavior using semantic outcomes and trajectory diagnostics. These signals include task progress, success rate, terminal reward trigger rate, stable landing evidence, safe contact evidence, episode length, contact instability, and reward hacking indicators.

Scalar scores alone cannot explain why a reward failed. A policy may obtain high dense progress reward while never triggering terminal success, or exploit contact-related rewards through unstable repeated events. To diagnose such failures, EG-RSA computes per-component reward attribution from logged component rewards. The framework identifies the dominant reward component, maps it to a semantic role, and combines this attribution with failure-mode diagnostics.

For example, if a dense approach component dominates total reward while terminal success remains zero, the policy may be exploiting dense guidance without completing the task. Such a diagnosis provides the LLM edit agent with evidence about which reward roles should be weakened, preserved, or complemented by new event rules.

3.4 Experience Memory and Edit Planning

EG-RSA maintains a memory store of reward-search experience. After each edit is evaluated, the framework records an outcome lesson

$$\ell_k = (S_k, \Delta_k, S'_k, O_k, O'_k, F_k, d_k), \tag{5}$$

where O_k and O'_k are the semantic outcomes before and after the edit, F_k contains failure-mode diagnostics, and d_k is the accept, reject, or rollback decision. If an edit improves task behavior or reduces risk, it is stored as an effective lesson. If an edit causes regression or increases reward-hacking risk, it is stored as a regression lesson.

At iteration k, EG-RSA retrieves relevant lessons from memory and includes them in the LLM edit prompt together with the current schema, semantic outcomes, failure modes, and attribution report. This allows the LLM to condition its proposed edit on accumulated reward-search history rather than only on the current scalar score.

3.5 Risk-Aware Validation and Rollback

Every proposed edit is validated before being applied. EG-RSA first checks operator legality and schema consistency. It then performs scale validation to avoid destabilizing weight changes. Finally, behavior risk audit examines whether the edit may amplify dense reward dominance, overemphasize terminal rewards before sufficient evidence, remove important stabilizing components, or resemble a prior regression lesson.

Edits are classified by risk level. High-risk edits are blocked or sent to repair. Low-risk edits proceed. Medium-risk edits are controlled by an audit policy. A strict policy may block medium-risk edits when success evidence is weak, while a relaxed policy can allow them with warnings and rely on later outcome evaluation and rollback. This distinction is important because overly conservative audit can suppress the exploration needed to escape low-success regimes.

After training a candidate schema, EG-RSA applies an outcome acceptor. If the candidate improves the relevant semantic metrics and does not increase hacking risk, the candidate schema becomes the next schema. Otherwise, the system rolls back to a previous best schema and stores the failed edit as a regression lesson.

3.6 Algorithm Summary

Algorithm 1 summarizes EG-RSA. The framework forms a closed loop between policy training, semantic diagnosis, memory retrieval, LLM edit planning, and risk-aware validation.

4 Experiments

4.1 Setup

We evaluate EG-RSA on LunarLander-v3, a continuous control task in the Gymnasium suite [Towers et al., 2024]. The agent must land a lunar module smoothly on a landing pad with low velocity,

Algorithm 1 EG-RSA Reward Schema Search

Require: Initial schema S_0, task description T, diagnostic specification D, memory store $\mathcal{L}$, iterations K

for $k = 0$ to $K - 1$ **do**
  Compile reward function R_{S_k} from schema S_k
  Train policy π_k using RL under reward R_{S_k}
  Collect semantic outcome O_k and trajectory diagnostics F_k
  Compute semantic reward role attribution A_k
  Retrieve relevant outcome lessons $\mathcal{L}_k$ from memory
  Construct edit prompt using $(T, S_k, O_k, F_k, A_k, \mathcal{L}_k)$
  Query LLM edit agent to produce edit plan Δ_k
  Validate Δ_k with operator constraints and scale audit
  Run behavior risk audit using $(F_k, A_k, \mathcal{L}_k)$
  if Δ_k passes validation and audit **then**
    $S'_k \leftarrow \text{Apply}(S_k, \Delta_k)$
    Train and evaluate candidate schema S'_k
    if candidate outcome is accepted **then**
      $S_{k+1} \leftarrow S'_k$
      Store effective outcome lesson in $\mathcal{L}$
    else
      $S_{k+1} \leftarrow$ rollback schema
      Store regression lesson in $\mathcal{L}$
    end if
  else
    Repair Δ_k or continue with S_k
    $S_{k+1} \leftarrow S_k$
  end if
end for
return Best schema $S^\star$

upright orientation, and stable ground contact (8-dim observation, 2-dim continuous action). All experiments use PPO [Schulman et al., 2017] with a fixed 1M-step training budget per iteration, running for 10 iterations with a DeepSeek-based LLM edit agent. All EG-RSA components are enabled: memory, attribution, hack detection, operator constraints. The oracle reward is used only for post-hoc evaluation. The default audit policy is strict.

4.2 Main Experiment and Case Studies

Table 1 presents the 10-iteration run under strict audit. The experiment exhibits three phases: an effective edit (Iter 0→1), a regression (Iter 1→2), and a prolonged low-success plateau (Iter 2–8). We examine each as a case study.

Effective edit (Iter 0→1). At iteration 0, the initial schema produces a policy achieving task score 0.478. Attribution identifies `r_approach_region` (dense guidance) as dominant (ratio 0.42). Two failure modes are detected: shaping-goal mismatch (no terminal success despite progress) and repeated event exploitation (rapid leg-contact toggling, 370 events/episode average). The LLM, informed by this diagnosis and having no prior lessons (first iteration), proposes: increase `r_landing_quality` weight, add a stable-landing event rule, decrease `r_approach_region`.

Table 1: EG-RSA 10-iteration run on LunarLander-v3 under strict audit.

Iter	Task	Semantic	Hack	Failures	Dominant Component
0	0.478	0.478	0.40	REE, SGM	r_approach_region
1	0.867	0.950	0.00	(none)	r_landing_quality
2	0.412	0.412	0.40	REE, SGM	r_approach_region
3	0.416	0.416	0.40	REE, SGM	r_landing_quality
4	0.367	0.367	0.40	REE, SGM	r_landing_quality
5	0.631	0.631	0.40	REE, SGM	r_approach_region
6	0.477	0.477	0.40	REE, SGM	r_landing_quality
7	0.422	0.422	0.20	SGM	r_landing_quality
8	0.665	0.665	0.40	REE, SGM	r_landing_quality

REE = repeated_event_exploitation; SGM = shaping_goal_mismatch.

Table 2: EG-RSA under relaxed audit.

Iter	Task	Semantic	Hack	Failures	Dominant Component
0	0.478	0.478	0.40	REE, SGM	r_approach_region
1	0.511	0.511	0.40	REE, SGM	r_approach_region
2	2.490	3.907	0.00	(none)	r_landing_quality
3	2.953	4.453	0.00	(none)	r_landing_quality

The edit is classified medium-risk but permitted as a first-iteration exception. After training, task score reaches 0.867, `r_landing_quality` becomes dominant, and both failure modes are absent. The system records an effective-edit lesson.

Regression and audit deadlock (Iter 1→8). At iteration 2, the task score drops to 0.412, failure modes return, and `r_approach_region` again dominates. The system records a regression lesson (metric delta −0.455). Following this regression, the LLM repeatedly proposes edits that strengthen terminal success rewards and weaken dense guidance—the same strategy that succeeded at iteration 1. However, these edits are now classified as medium risk (modifying terminal reward structure), and success evidence is weak (no terminal success since iteration 1). Under the strict audit policy, medium-risk edits with weak success evidence are blocked. The repair loop cannot resolve this: repairs adjust weights but cannot eliminate the risk classification. The result is a deadlock across iterations 2–8: scores remain between 0.367–0.665, failure modes persist, and the system cannot escape.

This deadlock is not an implementation flaw—it reveals a fundamental design tension. The same audit mechanism that prevents harmful edits can also block the exploration needed to establish success evidence. The system needs to modify terminal rewards to discover successful landing, but those modifications are precisely what strict audit blocks when success evidence is absent.

4.3 Relaxed Audit Policy

To test whether the deadlock is audit-induced, we run a second experiment with a relaxed policy: medium-risk edits under weak evidence are accepted with a warning. All other settings are identical.

Starting from the same initial schema (0.478), the relaxed-audit run follows a different trajectory. After a modest first edit (0.511), iteration 2 achieves a breakthrough: task score 2.490, no

failure modes, `r_landing_quality` dominant. Iteration 3 continues to improve (2.953). These results confirm the deadlock was audit-induced: the same search mechanism—attribution, memory, constrained editing—produces sustained improvement once the audit barrier is lowered.

These results do not imply audit should be removed. Rather, they motivate a risk-budget mechanism: high-risk edits remain blocked, medium-risk edits are permitted with monitoring, and rollback serves as a safety net. Designing such a mechanism is future work.

5 Discussion

These experiments serve as mechanism verification, not a performance benchmark. Three findings emerge. First, attribution-guided editing with structured memory can produce effective reward modifications, as demonstrated by the iteration 0→1 case study. Second, regression is real in reward search and the system tracks it through structured outcome lessons. Third, the audit deadlock reveals a fundamental tension: integrating safety constraints into reward search creates a tradeoff between risk prevention and exploration, where strict blocking can suppress the edits needed to establish success evidence.

Our study has several limitations. We evaluate on a single environment (LunarLander-v3). Task metrics are template-instantiated rather than auto-discovered. Audit rules are hand-designed for this environment. Experiments run for 10 iterations; longer-horizon dynamics remain unexplored. We do not benchmark against EUREKA or other methods—such comparison requires multi-environment evaluation beyond this mechanism-verification scope.

The audit deadlock finding has implications beyond EG-RSA. Any LLM-based reward design system that introduces safety constraints—rule-based, learned, or ensemble-based—faces the same tradeoff. Strict blocking of medium-risk changes under weak evidence can suppress the edits needed to establish that evidence in the first place.

Future work includes multi-environment evaluation, learned audit rules that adapt to environment-specific failure modes, implementation of a risk-budget mechanism, and integration of process-level reward models [Zheng et al., 2025] to refine attribution granularity.

6 Conclusion

We reformulated LLM-assisted reward design as experience-guided schema search, introducing semantic role attribution, structured outcome memory, and integrated risk audit with rollback. Mechanism-verification experiments on LunarLander-v3 demonstrate effective editing, regression tracking, and an audit-induced deadlock that reveals a safety–exploration tradeoff. Resolving this deadlock through relaxed audit motivates future work on risk-budget mechanisms for safe reward search.

References

Pieter Abbeel and Andrew Y. Ng. Apprenticeship learning via inverse reinforcement learning. In *Proceedings of the Twenty-first International Conference on Machine Learning*, 2004.

Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in ai safety. *arXiv preprint arXiv:1606.06565*, 2016. URL `https://arxiv.org/abs/1606.06565`.

Yuji Cao, Huan Zhao, Yuheng Cheng, Ting Shu, Yue Chen, Guolong Liu, Gaoqi Liang, and Junhua Zhao. A survey on large language model-enhanced reinforcement learning: Concept, taxonomy, and methods. *arXiv preprint arXiv:2404.00282*, 2024. URL https://arxiv.org/abs/2404.00282.

Hao Li, Xue Yang, Zhaokai Wang, Xizhou Zhu, Jie Zhou, Yu Qiao, Xiaogang Wang, Hongsheng Li, Lewei Lu, and Jifeng Dai. Auto mc-reward: Automated dense reward design with large language models for minecraft. *arXiv preprint arXiv:2312.09238*, 2023. URL https://arxiv.org/abs/2312.09238.

Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-level reward design via coding large language models. *arXiv preprint arXiv:2310.12931*, 2023. URL https://arxiv.org/abs/2310.12931.

Andrew Y. Ng and Stuart Russell. Algorithms for inverse reinforcement learning. In *Proceedings of the Seventeenth International Conference on Machine Learning*, pages 663–670, 2000.

Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning*, pages 278–287, 1999.

Alexander Pan, Kush Bhatia, and Jacob Steinhardt. The effects of reward misspecification: Mapping and mitigating misaligned models. *arXiv preprint arXiv:2201.03544*, 2022. URL https://arxiv.org/abs/2201.03544.

Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. *arXiv preprint arXiv:2304.03442*, 2023. URL https://arxiv.org/abs/2304.03442.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. URL https://arxiv.org/abs/1707.06347.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*, 2023. URL https://arxiv.org/abs/2303.11366.

Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward hacking. *Advances in Neural Information Processing Systems*, 35, 2022. URL https://arxiv.org/abs/2209.13085.

Shengjie Sun, Runze Liu, Jiafei Lyu, Jing-Wen Yang, Liangpeng Zhang, and Xiu Li. A large language model-driven reward design framework via dynamic feedback for reinforcement learning. *arXiv preprint arXiv:2410.14660*, 2024. URL https://arxiv.org/abs/2410.14660.

Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018.

Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024. URL https://arxiv.org/abs/2407.17032.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023. URL https://arxiv.org/abs/2305.16291.

Tianbao Xie, Siheng Zhao, Chen Henry Wu, Yitao Liu, Qian Luo, Victor Zhong, Yanchao Yang, and Tao Yu. Text2reward: Reward shaping with language models for reinforcement learning. *arXiv preprint arXiv:2309.11489*, 2023. URL https://arxiv.org/abs/2309.11489.

Congming Zheng, Xiangyu Zhu, et al. A survey of process reward models: From outcome signals to process supervisions for large language models. *arXiv preprint arXiv:2510.08049*, 2025. URL https://arxiv.org/abs/2510.08049.

A Appendix: Extended Experiment Details

A.1 Environment and Hyperparameters

LunarLander-v3 provides an 8-dimensional observation space (position, velocity, angle, angular velocity, leg contact) and a 2-dimensional continuous action space (main engine, side engines). PPO hyperparameters: learning rate 3×10^{-4}, 64 parallel environments, 256-step rollout, 10 epochs per update, GAE $\lambda = 0.95$, clipping $\epsilon = 0.2$. Training budget: 10^6 environment steps per iteration.

A.2 Experiment Mode

All EG-RSA components are enabled: `use_memory=true`, `use_attribution=true`, `use_hack_detector=true`, `use_llm_edit=true`, `use_operator_constraints=true`, `free_rewrite=false`. The LLM edit agent is DeepSeek-based.

A.3 Extended Iteration Traces

Full iteration data including per-iteration edit plans, audit reports, repair logs, and outcome lesson cards are available in the experiment directories under `experiments/eg_rsa_lunar_lander_v1_role_attrib_10x1` and `experiments/eg_rsa_lunar_lander_v1_role_attrib_10x1m_audit_relaxed/`.

A.4 Planned Ablations

Planned experiments include: disabling memory retrieval (feed-forward baseline), disabling attribution (scalar-feedback-only baseline), and removing operator constraints (free-code baseline). These ablations will isolate the contribution of each mechanism. Configurations exist in the `configs/` directory; experimental results are pending.